

CUDA PROGRAMMING FOR NVIDIA H100S

多 GPU

Prateek Shukla · 简体中文版

让我们用 **H100** 在 **10T token** 上训练一个 **1T** 参数模型

H100 实证观察表明，训练数据中每个 token、每个参数大约需要 6 次运算（FLOPs）（前向传播 + 反向传播 + 更新）。

总 FLOPs = $6 \times (10^{12}) \times (10^{13}) = 6 \times 10^{25}$ FLOPs

假设我们每秒能拿到 1000tflops

那大约是 1900 年！！

这就是为什么我们需要多块 GPU

多 GPU

与其花费数千年等待运算完成，我们可以直接把多块 GPU 连接在一起，获得更高吞吐量并运行更大的模型。这由两项技术支撑

NVLink Gen 4

绕开 CPU 的直接 GPU 到 GPU 互连。

900 GB/s 双向带宽。

比 PCIe Gen 5 快 7 倍。

NVSwitch

让节点中的每块 GPU 都能以全速与其他任何 GPU 通信

H100 集群与旧式集群对比

在传统计算中，GPU 是通过较慢的 PCIe 通道通信的离散单元。H100 范式改变了这一点：它创造了一个"网格（mesh）"，其中每块 GPU 都能以极高的速度访问其他任何 GPU 的内存。

一个 8 GPU 集群可以表现得好像拥有单一的内存池和计算核心池

这种架构让系统可以从单台服务器中的 8 块 GPU，扩展到 256 块 GPU 的集群（使用 NVLink Switch System），并以原生芯片速度通信。

H100 DGX 系统或一个节点

它配备 8 块采用 SXM5 形态的 NVIDIA H100 GPU。

该板载有 4 颗第三代 NVSwitch

GPU 与交换机通过第四代 NVLink 连接，为每块 GPU 提供高达 900 GB/s 的带宽

8 张网络接口卡（NIC）：NVIDIA ConnectX-7，一种专门设计用于把网络任务从主 CPU 卸载的处理器。

4x OSFP (Octal Small Form-factor Pluggable) 笼位

Nvidia H100 Superpod

一台 NVIDIA DGX H100 SuperPod 大约有 127-128 块 GPU，组织为 4 个 SU（Scalable Unit，可扩展单元），每个 SU 拥有 32 个 DGX H100 节点。

数字 32 使得使用标准交换机端口数（通常每台交换机 64 个端口）的"Level 1"网络达到完美均衡。

H100 SuperPod 的设计基于这样一个事实：单个 DGX H100 节点拥有 8 个独立的 NVIDIA ConnectX-7 网络接口（HCA）用于计算，每块 GPU 一个。

Nvlink

每块 H100 GPU 具有 900 GB/s 的双向带宽。这大约比用于连接 GPU 与 CPU 的标准 PCIe Gen5 接口快 7 倍。Nvlink 让 GPU 利用这一点在自己的 HBM 之间通信，并跳过 CPU 路径。

H100 使用 18 条独立的 NVLink"链路"来达到这一速度。与以往世代不同，第四代 NVLink 更注重密度优化，每条链路只使用两对高速差分对（从四对减少），从而能在同样的空间里塞进更多链路。

Nvswitch 3

在标准的 8 GPU HGX 板上，使用四颗 NVSwitch 芯片连接全部八块 H100 GPU。每台交换机连接 4 - 5 个 Nvlink 端口

这些交换机构成完全无阻塞的互连。这意味着每块 GPU 都能同时以 900 GB/s 的全速与任何其他 GPU 通信，而无需排队等待数据通道腾空

NVIDIA 把 NVSwitch 芯片放进称为 NVLink Switch System 的外部托盘。这些系统把多达 256 块 H100 GPU (32 个服务器机架) 连接成单个"SuperPOD"

NVSwitch 本身带有 ALU，可以在交换机内部执行加法/归约，这是带来巨大性能提升的最重要特性之一。

Connectx

在服务器机箱内部，GPU 通过 NVLink (900 GB/s) 交谈。一旦数据需要离开机箱去往另一台服务器，它就会撞上 ConnectX-7。每台 DGX 还有 8 个计算网络接口 (ConnectX-7)。

这就是瓶颈。你的训练速度取决于你管理这次 18 倍带宽骤降的效率。ConnectX-7 的存在就是为了把离开机箱的代价降到最小。

ConnectX7 让网络可以在 CPU 不知情的情况下读写 GPU 内存。

NIC 与网络交换机协作，在传输途中对梯度求和。这把网络流量从 $O(N)$ (随集群规模线性增长) 变为 $O(1)$ (恒定流量)。这是大规模训练能够线性扩展的唯一原因

OSFP 笼位

这 4 个物理笼位通过"双端口 (Twin-port)"技术提供 8 条独立的 400Gb/s 网络链路 (合计 3.2 Tb/s 带宽)。它们在内部连接到 8x NVIDIA ConnectX-7 网卡。

这使得 GPU Direct RDMA 成为可能, 让 GPU 能与其他 DGX 节点中的 GPU 交谈, 而不用给系统 CPU 增加负担。

这 4 个 OSFP 笼位专用于计算 fabric (Compute Fabric)。存储流量绝不经那些线缆。

存储 **fabric**

它帮助把海量数据集加载进 GPU 并保存 checkpoint。

它使用位于机箱背面标准 PCIe 插槽中的独立 PCIe 卡，而不是 OSFP 笼位。

这通常是胖树或标准的 leaf-spine 网络。与"rail"网络不同，任何 DGX 节点都需要能够与任何存储阵列交谈。

rail 对齐系统

在标准网络中，一台服务器可能只用一根线缆承载其所有组件的流量。在 DGX SuperPOD 中，网络在物理上被拆分为 8 个并行的、相互隔离的网络——这些就是"rail"。

一个节点内部有 8 块 GPU (编号 0 到 7)。Rail 1 把集群中每个节点的 GPU 0 连接到同一组交换机。Rail 2 把每个节点的 GPU 1 连接到另一组交换机。依此类推

在 leaf switch 层面，Rail 1 上的流量绝不会干扰 Rail 2 上的流量。这在整个集群中创造了 8 个独立的连接平面。

rail 系统利用 NVIDIA 的 SHARP 技术，把数据操作卸载到网络交换机本身。

rail 优化的胖树

SuperPOD 使用三层层级（Leaf、Spine、Core）把各个 SU 连接在一起

第 1 层：Leaf 层（与节点的连接）

第 2 层：Spine 层（SU 之间的连接）

第 3 层：Core / Super-Spine 层（最大规模）

这些层不过是一堆连接在一起的 Quantum-2 InfiniBand 交换机

Quantum-2 QM9700 InfiniBand 交换机

它提供超高带宽、低延迟的 GPU-GPU 互连

它拥有 64 个 400Gb/s 连接端口

当你看向交换机正面时，会看到 32 个笼位。它们采用 OSFP (Octal Small Form-factor Pluggable) 标准。每个 OSFP 笼位实际承载两条独立的 400Gb/s 链路。32 笼位 x 2 链路 = 64 逻辑端口。

由于 QM9700 有 64 个端口，它正好是一个 32 节点 SU 的完美尺寸。

32 个端口（下行）：连接到 SU 中的 32 个节点（例如 Rail 1 连接到全部 32 个节点上的 GPU 0）。

32 个端口（上行）：向上连接到 spine switch（第 2 层），与其他 SU 交谈。

leaf 层

这里就是线缆从 DGX H100 服务器背面物理引出的地方。

rail 隔离 (Rail Segregation) : 有 8 组独立的交换机"平面" :

Rail 1 交换机 : 只连接每个节点的第 1 张网卡 (GPU 0)。

Rail 8 交换机 : 只连接每个节点的第 8 张网卡 (GPU 7)。

leaf switch 处理本地可扩展单元 (Scalable Unit) 内的流量。如果节点 1 上的 GPU 0 需要与节点 2 (同一 SU 内) 上的 GPU 0 交谈, 流量路径是节点 -> Leaf Switch -> 节点。它从不需要沿链条走向更高的层级。

QM9700 使用 SHARPV3, 其能力是上一代的 32 倍, 使它能够处理复杂的 AI 数学。

leaf 层中的自适应路由

在 leaf 层，自适应路由对上行流量至关重要。

当数据包从一块 GPU 到达 leaf switch、并且需要前往另一个 SU 时，它必须上行到某台 spine switch。在无阻塞甚至超额订阅的胖树中，有多台可用的 spine switch 供它选用。

Quantum-2 交换机硬件不使用静态哈希（静态哈希总是把特定流发送到同一台 spine switch，可能造成碰撞），而是监视所有上行端口的队列深度与拥塞水平

交换机以逐包或逐消息的方式，动态地把数据包发送到最不拥挤的 spine switch 链路。这保证了即使通往 spine 的某条路径被堵死，流量也能顺畅地从其他路径流过。

spine 层

Spine 层把各台 Leaf switch 连接在一起。这使得 SU-1 中的节点可以与 SU-2 中的节点交谈。

Spine 组 : spine switch 也被组织成与 rail 对齐的组。

Spine 组 1 : 只连接处理 Rail 1 的 Leaf switch。

隔离 : 这保证 Rail 1 的流量绝不会意外"泄漏"到 Rail 2 的线缆上 , 否则会造成拥塞 (阻塞) 。

如果 SU-1 中的 GPU 0 需要与 SU-2 中的 GPU 0 交谈 , 流量路径为 : 节点 (SU1) -> Leaf (Rail 1) -> Spine (组 1) -> Leaf (Rail 1) -> 节点 (SU2)

spine 层中的自适应路由

在 spine 层，自适应路由管理穿越网络核心的流量。

通常，在标准的两层胖树中，从某台特定 spine switch 到某个目标 leaf switch 只有唯一的"下行"路径。不过，如果存在并行链路，自适应路由在处理故障与多路径方面仍然至关重要

如果通往某台特定 leaf switch 的缓冲区已满，spine switch 之间会传递这种背压。Quantum-2 的自适应路由特性（如 SHIELD）帮助隔离这种拥塞，使其不会扩散到 spine 中其他未受影响的流量流。

Core / Super-Spine 层（最大规模）

对于超大规模集群（例如 127 个节点或更多），会加入第三层。

功能：这一层把多个"pod"（SU 的集群）连接在一起。

光模块：在这一层，系统通常改用 800G 光收发器以减少所需线缆数量，同时在逻辑上再拆分回两条 400G 链路。

SHIELD

在传统 InfiniBand 网络中，如果线缆断裂或链路抖动，软件控制器需要大约 5 - 30 秒来计算新的路由表并下发给所有交换机。这会让整个训练运行崩溃

SHIELD 把这套恢复逻辑直接搬进交换机硬件 ASIC。

当一台交换机看到某条链路断开时，它不会只是丢包。它立即在自己的本地硬件表中检查是否存在替代的有效路径。如果找到，它就更新自己的表以绕开出问题的节点、转向健康的邻居；如果失败，它就向该邻居发送硬件信号，使今后不会有任何流量再发给它。

GPU 间通信的 **P2P** 机制

CUDA 的 P2P (点对点 , Peer-to-Peer) 机制是一种允许两块 GPU 在不借助 CPU 的情况下通信的特性

我们可以显式地做 P2P 内存拷贝 , 或者获得 P2P 直接访问 , 整个传输都走 Nvlink。

统一虚拟寻址 (Unified Virtual Addressing) 支撑起这整个系统

统一虚拟寻址（**UVA**）

UVA 让 CPU 和 GPU 共享单一的虚拟地址空间。

在 UVA 之前：CPU 有自己的内存指针，GPU 也有自己的。你必须手动管理哪个指针指向哪块物理内存。

有了 UVA：系统仅凭指针值本身，就能确定数据在物理上的确切位置（在系统 RAM 中，还是在 H100 的 HBM3 内存上）。

你需要用 `cudaDeviceEnablePeerAccess` 启用对等访问，否则这个特性无法工作

cudaDeviceEnablePeerAccess

如果没有这个调用，GPU 可能不会使用 Nvlink，而是走 PCIe

几个要点

`cudaDeviceEnablePeerAccess(peerDevice, 0)` 是单向的。如果你需要来回拷贝，或者两侧都要有 kernel 访问内存，你必须在两块设备上都调用它。

如果不启用它，你可能会退回到较慢的 PCIe 路径

在 H100 服务器上，如果可用，这个函数会自动使用 NVLink (900 GB/s)；否则，它使用 PCIe Gen5

cudaMemCpyPeer

让数据可以在两块独立 GPU 的内存之间直接传输，而不涉及主机（CPU）内存

要使用 Nvlink 进行拷贝，必须先启用 `cudaDeviceEnablePeerAccess`

裸 P2P 的真相

在 H100 HGX 板上，你面对的并不是 8 块连成一条线的 GPU，而是一张通过 NVSwitch 连接的复杂网格。

P2P：你必须手动管理自己正在穿越哪条链路。如果 GPU 0 与 GPU 7 交谈，它是直连吗？还是经由 GPU 3 跳转（hop）？

H100 拥有 900 GB/s 的双向 NVLink 带宽。如果你从某个 SM 跨越 NVLink fabric 发出标准的 LD/ST（Load/Store）指令，你很可能只在使用单条独立的 NVLink 通道或其中一部分。你会很难打满那条管道。这就像用一根吸管去喝消防水带里的水。

正是因为这些原因，我们需要像 NCCL 这样的库来管理多 GPU 连接